Smart Integration Testing

Using Cucumber Profiles
&
Analyzing code changes

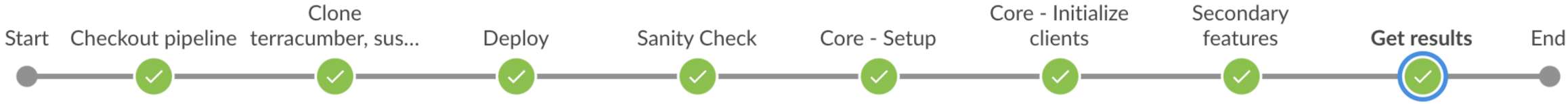
Óscar Barrios
Test Automation Specialist @SUSE

1. Continuous Integration Tests
2. Testing our Pull Request
3. Smart testing

CI Testing

Full regresstion tests

1. Check out master branch
2. Run all the tests using a full environment
3. The QA Squad review a big Test Report



smdba
spacecmd
Visualization
Notification messages
Virtual host manager
Subscription matching
formulas
SP Migration
CVE Audit
Kubernetes Integration
Power Management
Onboarding
Content life-cycle
Recurring Actions
Maintenance Windows
Cluster Management
Building Container Images
Openscap
Ubuntu
Action Chains
Content Staging
Proxy
Salt
Salt SSH
Tomcat
Changing Software Channels
Monitoring
Cobbler
Traditional Client
Virtualization
Hub
Retail

Test time: 7-9 hours

Testing our Pull Request

Before we merge the code changes into master branch,
What about be sure that we don't break CI Tests?

What did you change on your branch?

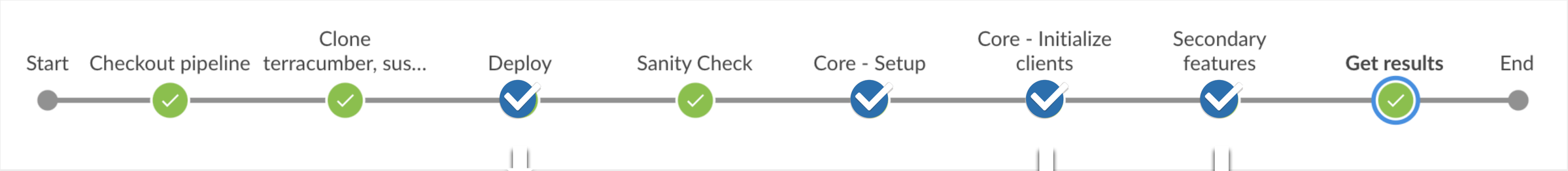
On my Pull Request…

"I added power management support for Ubuntu clients, including a change in the bootstrap mechanism."

Testing our Pull Request

Before we merge the code changes into master branch,
What about be sure that we don't break CI Tests?

1. Check out Pull Request branch
2. Test affected scopes on a custom test environment
3. The PR's owner review a focused Test report



1 Deploy only Ubuntu Clients

2 Onboarding only Ubuntu Clients

3 Power management for Ubuntu

Test time: 1-2 hours

Smart testing

Don’t use a sledgehammer to crack a nut

- We want to test only the parts of the product that could be affected by the code changes of our branch
- In some cases, your code changes only affect to a sub-set of clients, let’s deploy only the machines we will use
- In most of the cases, the code changes can tell us wich functional areas can be affected
- We tag each test file, so we identify which functional area is testing

“Deploy and test only what you suspect could be compromised.”

Smart testing

Don’t use a sledgehammer to crack a nut

Cucumber Profiles

You can specify configuration options for Cucumber in a **cucumber.yml**, defining a profile requires a name and then the **command-line options** that you want to execute.

We will use **Cucumber profiles to map them with Cucumber Tags**, Cucumber will only run the tests prepended with the tag.

We will have a profile and tag for each functional area, so we can **filter our tests by functional areas**.

testsuite 〉 config 〉 cucumber.yml

cucumber.yml

```yaml
default: -f pretty -r features
smdba: --tags @scope_smdba
spacecmd: --tags @scope_spacecmd
spacewalk_utils: --tags @scope_spacewalk_utils
visualization: --tags @scope_visualization
notification_message: --tags @scope_notification_message
virtual_host_manager: --tags @scope_virtual_host_manager
subscription_matching: --tags @scope_subscription_matching
formulas: --tags @scope_formulas
sp_migration: --tags @scope_sp_migration
cve_audit: --tags @scope_cve_audit
onboarding: --tags @scope_onboarding
content_lifecycle_management: --tags @scope_content_lifecycle_management
res: --tags @scope_res
recurring_actions: --tags @scope_recurring_actions
maintenance_windows: --tags @scope_maintenance_windows
cluster_management: --tags @scope_cluster_management
building_container_images: --tags @scope_building_container_images
kubernetes_integration: --tags @scope_kubernetes_integration
openscap: --tags @scope_openscap
ubuntu: --tags @scope_ubuntu
action_chains: --tags @scope_action_chains
salt_ssh: --tags @scope_salt_ssh
tomcat: --tags @scope_tomcat
```

Smart testing

Don’t use a sledgehammer to crack a nut

Cucumber Profiles

When we run the Jenkins Job, we must provide the cucumber profiles we want to include.

At that moment, **our Jenkins jobs allow to select a concrete Cucumber profile manually**, if you don’t select any, the default profile runs all the tests.

```ruby
namespace :cucumber do
  Dir.glob(File.join(Dir.pwd, *smth ['run_sets', '*.yml'] )).each do |entry|
    Cucumber::Rake::Task.new(File.basename(entry, '.yml').to_sym) do |t|
      filename = File.basename(entry, '.yml').to_sym
      timestamp = Time.now.strftime('%Y%m%d%H%M%S')
      json_results = "-f json -o output_#{timestamp}-#{filename}.json"
      html_results = "-f html -o output_#{timestamp}-#{filename}.html"
      profile = ENV['PROFILE'] || 'default'
      cucumber_opts = %W[#{html_results} #{json_results} #{junit_results} -f rerun --profile #{profile}
      features = YAML.safe_load(File.read(entry))
      t.cucumber_opts = cucumber_opts + features unless features.nil?
    end
  end
end
```

functional_scope

default

Choose a functional scope and your job will only run tests of that scope

Build

4 In-progress

Choose or suggest automatically the functional areas affected by the code changes:

- By path (folder, filename, file extension, ...)
- By a comment on the header of the file
- Parsing the .changes, looking for a keyword
- As a guidance we can use the GitHub Action [Labeler]

Deploy only the clients we will use:

- For each Cucumber profile define which clients are necessary
- Filter resources from Terraform

Integrate the PR test in GitHub:

- Trigger OBS builds
- Pool of Test Environments to run PR tests in parallel
- Parametrizable main.tf pointing to OBS build
- GitHub action using a self-hosted runner